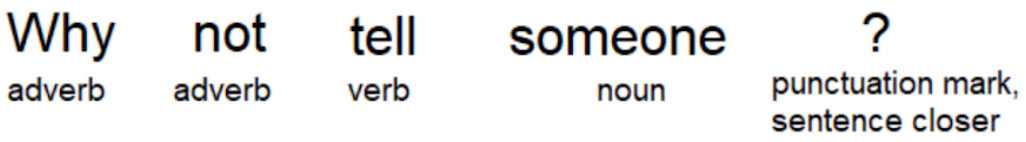
Name: Mohaiya aldossary
Student ID: 2230004579

Part-of-speech (POS)

Part-of-speech (POS) tagging is a popular NLP technique which refers to categorizing words in a text (corpus) in correspondence with a particular part of speech, depending on the definition of the word and its context.

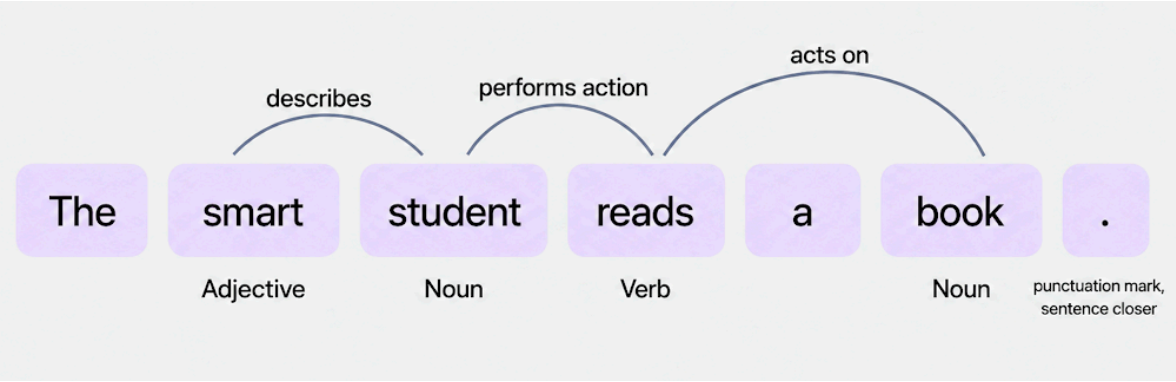


Benefits of POS tagging

- For some applications, we want to focus on certain POS:

E.g. Nouns are important for information retrival. Adjectives are important for sentiment analysis.

- POS Taggning is mostly useful in Named Entity Recognition (NER) applications and extracting relations between words.



POS Tagging can be used in:

- Grammar correction
- Spelling correction
- Question-Answering

POS tagging using NLTK

List of POS tags and their meaning in NLTK

Tag	Part of Speech	Example Words	Description
CC	Coordinating conjunction	and, but, or	Joins words or clauses
CD	Cardinal number	one, two, 100	Numbers
DT	Determiner	the, a, an, this	Specifies a noun
EX	Existential there	there	Indicates existence (“there is”)
FW	Foreign word	bonjour, gracias	Non-English word
IN	Preposition or subordinating conjunction	in, on, because	Links nouns or clauses
JJ	Adjective	happy, tall, blue	Describes a noun
JJR	Adjective, comparative	bigger, faster	Comparative adjective
JJS	Adjective, superlative	biggest, fastest	Superlative adjective
LS	List item marker	1., a., i.	Used in lists
MD	Modal	can, will, should	Auxiliary expressing mood
NN	Noun, singular or mass	dog, car, love	Common singular noun
NNS	Noun, plural	dogs, cars	Common plural noun
NNP	Proper noun, singular	John, London	Specific singular noun
NNPS	Proper noun, plural	Americans, Italians	Specific plural noun
PDT	Predeterminer	all, both, half	Comes before a determiner
POS	Possessive ending	’s, ’	Indicates possession
PRP	Personal pronoun	I, you, he, she	Replaces a noun
PRP$	Possessive pronoun	my, your, his	Shows possession
RB	Adverb	quickly, very	Modifies verb, adjective, or adverb
RBR	Adverb, comparative	faster, better	Comparative adverb
RBS	Adverb, superlative	fastest, best	Superlative adverb
RP	Particle	up, off	Forms phrasal verbs
SYM	Symbol	$, %, +	Mathematical or other symbols
TO	to	to	Infinitive marker or preposition
UH	Interjection	wow, hey	Expresses emotion
VB	Verb, base form	run, eat, go	Base form of verb
VBD	Verb, past tense	ran, ate, went	Past tense verb
VBG	Verb, gerund/present participle	running, eating	-ing form
VBN	Verb, past participle	eaten, gone	Past participle
VBP	Verb, non-3rd person singular present	run, eat	Present tense (I/you/we/they)
VBZ	Verb, 3rd person singular present	runs, eats	Present tense (he/she/it)
WDT	Wh-determiner	which, that	Introduces a clause
WP	Wh-pronoun	who, what	Question or relative pronoun
WP$	Possessive wh-pronoun	whose	Possessive interrogative

pos_tag method

In [1]:

```
%pip -q install "nltk>=3.9,<4" "spacy>=3.8,<3.9"
```

```
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 23.2.1 -> 26.2.1
[notice] To update, run: C:\Users\ezio9\OneDrive\Desktop\IAU\NLP\NLP lab\.lab_work\venv\Scripts\python.exe -m pip install --
upgrade pip
```

In [2]:

```python
import nltk
nltk.download('averaged_perceptron_tagger_eng', raise_on_error=True)
nltk.download('punkt_tab', raise_on_error=True)
#nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\ezio9\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\ezio9\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Out[2]: True

In [3]:

```python
from nltk import pos_tag
from nltk.tokenize import word_tokenize

text = "I received my new phone broken from Amazon."
tokens = word_tokenize(text)

for word in tokens:
    print(f'word: ({word}), part of speech: ({pos_tag([word])[0][1]})')
```

```
word: (I), part of speech: (PRP)
word: (received), part of speech: (VBN)
word: (my), part of speech: (PRP$)
word: (new), part of speech: (JJ)
word: (phone), part of speech: (NN)
word: (broken), part of speech: (NN)
word: (from), part of speech: (IN)
word: (Amazon), part of speech: (NN)
word: (.), part of speech: (.)
```

POS tagging using Spacy

Coarse-grained: Tells us the general grammatical category of a word.

The	students	are	studying
DET	NOUN	AUX	VERB

Fine-grained: Gives us more specific grammatical information.

The	students	are	studying
DT	NNS	VBP	VBG
Determiner	Plural noun	Present-tense verb	Verb, -ing form

Coarse-grained Part-of-speech Tags

Every token is assigned a POS Tag from the following list:

POS	DESCRIPTION	EXAMPLES
ADJ	adjective	*big, old, green, incomprehensible, first*
ADP	adposition	*in, to, during*
ADV	adverb	*very, tomorrow, down, where, there*
AUX	auxiliary	*is, has (done), will (do), should (do)*
CONJ	conjunction	*and, or, but*
CCONJ	coordinating conjunction	*and, or, but*
DET	determiner	*a, an, the*
INTJ	interjection	*psst, ouch, bravo, hello*
NOUN	noun	*girl, cat, tree, air, beauty*
NUM	numeral	*1, 2017, one, seventy-seven, IV, MMXIV*
PART	particle	*'s, not,*
PRON	pronoun	*I, you, he, she, myself, themselves, somebody*
PROPN	proper noun	*Mary, John, London, NATO, HBO*
PUNCT	punctuation	*., (,), ?*
SCONJ	subordinating conjunction	*if, while, that*
SYM	symbol	*$, %, §, ©, +, −, ×, ÷, =, :), 😝 *
VERB	verb	*run, runs, running, eat, ate, eating*
X	other	*sfpksdpsxmsa*
SPACE	space	

Fine-grained Part-of-speech Tags

Tokens are subsequently given a fine-grained tag as determined by morphology:

Coarse-Grained Tag (`pos_`)	Fine-Grained Tag (`tag_`)	Description	Example(s)
ADJ	JJ	Adjective	blue, quick, large
	JJR	Comparative adjective	bigger, faster
	JJS	Superlative adjective	biggest, fastest
ADP	IN	Preposition or subordinating conjunction	in, on, because, after
ADV	RB	Adverb	quickly, very, too
	RBR	Comparative adverb	more, less
	RBS	Superlative adverb	most, least
	WRB	Wh-adverb	where, when, how
AUX	MD	Modal auxiliary	can, will, should, may
CCONJ	CC	Coordinating conjunction	and, but, or
DET	DT	Determiner	the, a, this, those
	PDT	Predeterminer	all, both, half
	WDT	Wh-determiner	which, that
INTJ	UH	Interjection	oh, wow, um, hey
NOUN	NN	Noun, singular or mass	cat, water, information
	NNS	Noun, plural	cats, dogs
NUM	CD	Cardinal number	one, 42, twenty
PART	POS	Possessive ending ('s)	's (as in "John's")
	RP	Particle	up, off

Coarse-Grained Tag (`pos_`)	Fine-Grained Tag (`tag_`)	Description	Example(s)
	TO	“to” (infinitive marker or preposition)	to (as in “to go”)
PRON	EX	Existential “there”	there (as in “there is”)
	PRP	Personal pronoun	I, you, he, she
	PRP$	Possessive pronoun	my, your, his, her
	WP	Wh-pronoun	who, what, whom
	WP$	Possessive wh-pronoun	whose
PROPN	NNP	Proper noun, singular	London, Mary
	NNPS	Proper noun, plural	Beatles, Americas
PUNCT	``	Opening quotation mark	“
	''	Closing quotation mark	”
	(	Left parenthesis	(
	)	Right parenthesis	)
	,	Comma	,
	:	Colon / semicolon / ellipsis	:, ;, …
	.	Sentence-final punctuation	., !, ?
	--	Dash / double hyphen	--
SYM	SYM	Symbol	%, +, =, &
	$	Dollar sign	$
	#	Number sign	#
VERB	VB	Verb, base form	eat, go, see
	VBD	Verb, past tense	ate, went, saw
	VBG	Verb, gerund/present participle	eating, going
	VBN	Verb, past participle	eaten, gone
	VBP	Verb, non-3rd person singular present	run, eat (I run)
	VBZ	Verb, 3rd person singular present	runs, eats
X	FW	Foreign word	déjà, d’où
	LS	List item marker	1), a.

In [4]:

```
import spacy
nlp = spacy.load('assets/en_core_web_sm')
```

In [5]:

```
doc = nlp(u"The quick brown fox jumped over the lazy dog's back.")
```

- To view the coarse POS tag use `token.pos_`
- To view the fine-grained tag use `token.tag_`
- To view the description of either type of tag use `spacy.explain(tag)`

In [6]:
```
for token in doc:
    print(f'{token.text:{10}} {token.pos_:{8}} {token.tag_:{6}} {spacy.explain(token.tag_)}')
```

```
The        DET      DT     determiner
quick      ADJ      JJ     adjective (English), other noun-modifier (Chinese)
brown      ADJ      JJ     adjective (English), other noun-modifier (Chinese)
fox        NOUN     NN     noun, singular or mass
jumped     VERB     VBD    verb, past tense
over       ADP      IN     conjunction, subordinating or preposition
the        DET      DT     determiner
lazy       ADJ      JJ     adjective (English), other noun-modifier (Chinese)
dog        NOUN     NN     noun, singular or mass
's         PART     POS    possessive ending
back       NOUN     NN     noun, singular or mass
.          PUNCT    .      punctuation mark, sentence closer
```

In the English language, the same word can have different meanings, even within the same sentence.

For this reason, morphology is important. **spaCy** uses machine learning algorithms to best predict the use of a token in a sentence. Is "*I read books on NLP*" present or past tense?

In [7]:
```
doc = nlp(u'I read books on NLP.')
r = doc[1]

print(f'{r.text:{10}} {r.pos_:{8}} {r.tag_:{6}} {spacy.explain(r.tag_)}')
```

```
read       VERB     VBP    verb, non-3rd person singular present
```

In [8]:
```
doc = nlp(u'I read a book on NLP.')
r = doc[1]

print(f'{r.text:{10}} {r.pos_:{8}} {r.tag_:{6}} {spacy.explain(r.tag_)}')
```

```
read       VERB     VBD    verb, past tense
```

What tag will be assigned if you added **'always'**?

In [9]:
```
doc = nlp(u'I always read a book on NLP.')
r = doc[2]

print(f'{r.text:{10}} {r.pos_:{8}} {r.tag_:{6}} {spacy.explain(r.tag_)}')
```

```
read       VERB     VBP    verb, non-3rd person singular present
```

Visualizing Parts of Speech

spaCy offers an outstanding visualizer called **displaCy**:

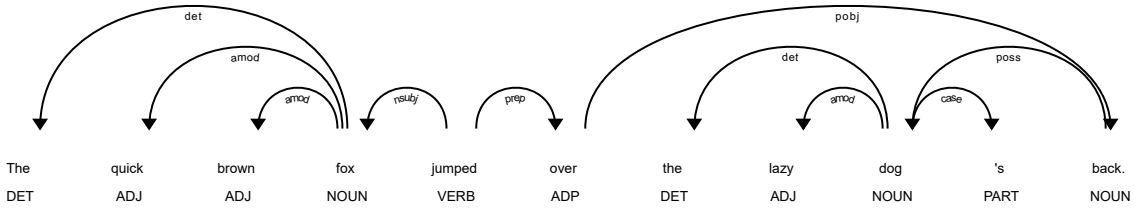
In [10]:
```
from spacy import displacy
```

In [11]:

```
# Create a simple Doc object
doc = nlp(u"The quick brown fox jumped over the lazy dog's back.")
```

In [12]:

```
# Render the dependency parse immediately inside Jupyter:
displacy.render(doc, style='dep', jupyter=True, options={'distance': 110})
```



Example: Find the adjective words in a sentance

In [13]:

```
from nltk import pos_tag
from nltk.tokenize import word_tokenize, sent_tokenize
```

In [14]:

```
text =  "Mobile is very handy. Packed with good performances and nice camera. This is my first iPhone. Very good experience with Apple. Battery is worst. I am hardly getting 5 hours screen on time. Positive thing is I came to know how many hours I was spending on my mobile. Because of poor battery now I am only using mobile for 4 hours a day."

print(text, end='\n\n')

for sent in sent_tokenize(text):
    tokens = word_tokenize(sent)
    part_of_speech = pos_tag(tokens)

    for i in part_of_speech:
        if 'JJ' in i:
            print(i)
    print('============================')
```

```
Mobile is very handy. Packed with good performances and nice camera. This is my first iPhone. Very good experience with Apple. Battery is worst. I am hardly getting 5 hours screen on time. Positive thing is I came to know how many hours I was spending on my mobile. Because of poor battery now I am only using mobile for 4 hours a day.

('handy', 'JJ')
============================
('good', 'JJ')
('nice', 'JJ')
============================
('first', 'JJ')
============================
('good', 'JJ')
============================
============================
============================
('Positive', 'JJ')
('many', 'JJ')
============================
('poor', 'JJ')
============================
```

Task 1

sen = nlp(u"I like watching football matches. I do not know how to play it though")

Use the spacy document object below to print out the following:

1. POS tag of the word "play". (Use sen[#].pos_)
2. Fine-grained POS tag for the word "play". (Use sen[#].tag_)
3. The meaning of the Fine-grained POS tag produced in (ii). (Use spacy.explain(sen[#].tag_)
4. Print the text, coarse-grained POS tags, fine-grained POS tags, and the explanation for the tags for all the words in the sentence.

Hint:

```
for word in sen:
 print(f'{word.text:{12}} {word.pos_:{10}} {word.tag_:{8}} {spacy.explain(word.tag_)}')
```

In [15]:

```
sen = nlp(u"I like watching football matches. I do not know how to play it though")

print(sen[12].pos_)
print(sen[12].tag_)
print(spacy.explain(sen[12].tag_))
print()
for word in sen:
    print(f'{word.text:{12}} {word.pos_:{10}} {word.tag_:{8}} {spacy.explain(word.tag_)}')
```

```
VERB
VB
verb, base form

I            PRON       PRP      pronoun, personal
like         VERB       VBP      verb, non-3rd person singular present
watching     VERB       VBG      verb, gerund or present participle
football     NOUN       NN       noun, singular or mass
matches      NOUN       NNS      noun, plural
.            PUNCT      .        punctuation mark, sentence closer
I            PRON       PRP      pronoun, personal
do           AUX        VBP      verb, non-3rd person singular present
not          PART       RB       adverb
know         VERB       VB       verb, base form
how          SCONJ      WRB      wh-adverb
to           PART       TO       infinitival "to"
play         VERB       VB       verb, base form
it           PRON       PRP      pronoun, personal
though       ADV        RB       adverb
```

Task 2

Compare the explanation of the words “google” and “table” in the following sentences.

1. “Did you search on Google?”
2. “Let us google it.”
3. “The has placed the food on the table”
4. “She plans to table a motion for debate in Parliament”

Hint: `word = sen[#]`

In [16]:

```
sentences = [
    "Did you search on Google?",
    "Let us google it.",
    "The has placed the food on the table",
    "She plans to table a motion for debate in Parliament",
]

for sentence in sentences:
    sen = nlp(sentence)
    print(sentence)
    for word in sen:
        if word.text.lower() in ("google", "table"):
            print(f'{word.text:{12}} {word.pos_:{10}} {word.tag_:{8}} {spacy.explain(word.tag_)}')
    print()
```

```
Did you search on Google?
Google       PROPN      NNP      noun, proper singular

Let us google it.
google       VERB       VB       verb, base form

The has placed the food on the table
table        NOUN       NN       noun, singular or mass

She plans to table a motion for debate in Parliament
table        VERB       VB       verb, base form
```

Task 3

Visualize the part of speech of each word in the sentence below.

“I like to play football. I hated it in my childhood though”

Hint: `displacy.render(doc, style='dep', jupyter=True, options={'distance': 110})`

In [17]:

```
doc = nlp(u"I like to play football. I hated it in my childhood though")
displacy.render(doc, style='dep', jupyter=True, options={'distance': 110})
```

